Contents

vmspawnd — Enterprise VM Management Platform

The Problem

Organizations running Linux infrastructure need a unified way to manage virtual machines. Existing solutions are either:

- **Too heavy** — VMware vSphere, Proxmox, and OpenStack require complex multi-server deployments, dedicated storage infrastructure, and specialized operations teams
- **Too basic** — Manual QEMU/KVM management with shell scripts doesn't scale and lacks security, monitoring, or multi-user access
- **Too locked-in** — Cloud-only solutions (AWS, Azure, GCP) create vendor dependency with unpredictable costs

vmspawnd fills the gap — a single-binary VM management platform that runs on any Linux server with systemd, providing enterprise features without enterprise complexity.

What Is vmspawnd?

vmspawnd is a production-grade virtual machine management platform built in Rust. It wraps systemd-vmspawn and systemd-machined with a complete management layer:

- **One binary, one config file, one systemd service** — deploys in under 5 minutes
- **480+ REST API endpoints** with JWT authentication, RBAC, and audit logging
- **5 management interfaces** — CLI, terminal UI, web dashboard, Kubernetes operator, Terraform provider
- **Enterprise features** — HA clustering, live migration, GPU passthrough, backup/restore, network policies

```
sudo systemctl enable --now vmspawnd
# That's it. Platform is running.
```

Key Differentiators

1. Built on systemd (Not a Custom Hypervisor)

vmspawnd leverages systemd-vmspawn and systemd-machined — the VM management tools built into every modern Linux distribution. This means:

- No custom kernel modules or hypervisor patches
- VMs are first-class systemd units with journal logging, socket activation, and watchdog support
- Works on Fedora, Ubuntu, Debian, RHEL, SUSE — any distro with systemd 256+
- Upstream-maintained VM lifecycle, not a fork

2. Single Binary, Zero Dependencies

vmspawnd	Proxmox	OpenStack
1 binary (15MB)	200+ packages	1000+ packages
1 config file	50+ config files	100+ config files
5 min setup	2+ hours	2+ days
Runs on any Linux	Debian only	Ubuntu/RHEL
SQLite user store	PostgreSQL required	MySQL + RabbitMQ + Memcached

3. Security-First Architecture

The entire codebase has undergone a **9-round security audit** with verified fixes:

- **Zero unsafe Rust** — memory-safe by construction
- **Zero shell pipelines** — all subprocess calls use safe argument passing
- **JWT + RBAC** on every API endpoint (Admin/User/Viewer roles)
- **bcrypt password hashing** with auto-generated secrets (0600 file permissions)
- **Rate limiting** on authentication (5 attempts/5 min)

- **Input validation** on every user-facing parameter
- **Audit logging** on all VM lifecycle operations
- **Path traversal protection** with canonicalization
- **SQL injection prevention** — all queries parameterized

4. Rust Performance

- **Sub-millisecond API response times** — async Axum + Tokio runtime
- **Low memory footprint** — ~20MB RSS for the daemon
- **No garbage collection pauses** — predictable latency
- **Safe concurrency** — per-VM mutexes prevent race conditions

---

Feature Overview

VM Lifecycle

- Create, start, stop, restart, pause, resume, delete
- Full and linked cloning with CoW support
- Templates for rapid deployment
- Declarative config via YAML (`vmctl apply -f config.yaml`)
- Multiple disk formats: qcow2, raw, vmdk, vdi

Storage

- **6 backends**: Local, NFS, LVM, LVM-thin, ZFS, Ceph/RBD
- Volume CRUD with attach/detach, resize, clone
- Snapshot create/restore with retention policies
- ZFS replication with incremental send/receive
- Ceph cluster health monitoring and RBD image management

Networking

- **Network Policies** — Cilium-style label-based ingress/egress rules
- **VM Firewall** — Per-VM firewall profiles and zones via nftables
- **Service Mesh** — Virtual IP load balancing (round-robin, least-conn, random, IP-hash)
- **QoS / Traffic Shaping** — Guaranteed/max rate, burst, priority-based bandwidth
- **DNS Policy** — Zone management, upstream servers, domain blocking
- **VPN Mesh** — WireGuard tunnels (point-to-point, hub-spoke, full-mesh)
- **Packet Mirror** — Traffic capture for debugging
- **NAT Gateway** — Masquerade, SNAT, DNAT, hairpin NAT
- **Network Monitor** — Per-VM bandwidth tracking with threshold alerts

Security

- JWT authentication with configurable token expiration
- 3-tier RBAC (Admin / User / Viewer) enforced on every endpoint
- API keys for service-to-service authentication
- TLS/HTTPS with certificate management
- Audit logging with JSON/CSV export
- Encryption at rest
- Rate limiting on authentication

High Availability

- etcd-based clustering with leader election
- Predictive DRS (Distributed Resource Scheduling)
- Fault tolerance with automatic failover and fencing
- Distributed storage replication
- Site recovery with failover/reprotect workflows

Monitoring & Automation

- Prometheus metrics exporter (`/metrics` endpoint)
- Analytics dashboard with historical data
- Multi-channel notifications: Email, Slack, Webhook, Microsoft Teams
- VM scheduling (once, daily, weekly)
- Backup/restore with retention policies and incremental backups
- Resource quotas, pools, and datacenter abstractions

Console Access

- WebSocket terminal via xterm.js (browser-based SSH)
- VNC graphical console via noVNC proxy
- Authenticated with same JWT tokens as API

Cloud & Virtualization

- cloud-init integration (NoCloud datasource)
- TPM/vTPM support (1.2 and 2.0 via swtpm)
- GPU passthrough (NVIDIA, AMD, Intel GVT-g)
- Live and offline VM migration
- CPU pinning and NUMA optimization

Management Interfaces

CLI (`vmctl`)

Scriptable command-line tool with JSON/YAML/table output:

```
vmctl list -o json
vmctl create myvm --image=ubuntu.qcow2 --cpus=4 --memory=4G
vmctl start myvm
vmctl apply -f infrastructure.yaml
vmctl policy list
vmctl ceph health my-pool
```

Terminal UI (`vmctl-tui`)

k9s-style dashboard with 8 views, vim keybindings, sparkline graphs, and live API data.

Web Dashboard

React-based UI with 37+ pages, command palette (Ctrl+K), dark theme, real-time WebSocket updates, and bulk operations.

Kubernetes Operator

Manage VMs as `VirtualMachine` CRDs with automatic reconciliation:

```yaml
apiVersion: vmspawnd.io/v1
kind: VirtualMachine
metadata:
  name: web-server
spec:
  image: ubuntu-22.04.qcow2
  cpus: 4
  memory: 4096
```

Terraform Provider

Declarative VM provisioning with full plan/apply workflow:

```
resource "vmspawnd_vm" "web" {
  name   = "web-server"
  image  = "ubuntu-22.04.qcow2"
  cpus   = 4
  memory = 4096
}
```

Architecture

```
         +-----------+    +-----------+    +-----------+
         |   vmctl   |    | vmctl-tui |    |  Web UI   |
         |   (CLI)   |    |   (TUI)   |    | (React)   |
         +-----+-----+    +-----+-----+    +----+------+
               |                |               |
     +---------+----------+     |      +---------+
     |         |          |     |      |
+----v----+   |   +------v-----v------v-------+
|   K8s   |   |   |       vmspawnd daemon      |
|Operator |   |   |   REST API + WebSocket      |
+---------+   |   +---+----+----+----+----+----+
               |       |    |    |    |    |
+-----------+  |       |    |    |    |    |
| Terraform |--+       |    |    |    |    |
| Provider  |         |    |    |    |    |
+-----------+         |    |    |    |    |
                       v    v    v    v    v
                systemd-vmspawn + systemd-machined
```

Deployment Models

Single Server

One Linux server running vmspawnd with local storage. Suitable for development, testing, small teams, and edge deployments.

Requirements: Linux with systemd 256+, 4GB RAM minimum

Multi-Node Cluster

Multiple vmspawnd nodes with etcd-based clustering, shared storage (NFS/Ceph), live migration, and HA failover.

Requirements: 3+ nodes, shared storage, etcd cluster

Kubernetes-Managed

vmspawnd nodes managed by the Kubernetes operator. VMs defined as CRDs alongside containerized workloads.

Requirements: Kubernetes cluster with vmspawnd operator deployed

---

Comparison

Feature	vmspawnd	Proxmox VE	OpenStack	libvirt/virsh
Single-binary deployment	Yes	No	No	N/A
REST API	480+ endpoints	~50	~200	XML-RPC
Web UI	Yes	Yes	Yes (Horizon)	No
CLI	Yes	Yes	Yes	Yes
Terminal UI	Yes	No	No	No
Kubernetes Operator	Yes	No	Yes	No
Terraform Provider	Yes	Yes	Yes	Yes
Network Policies	Cilium-style	Basic	Neutron	No
Service Mesh	Yes	No	No	No
VPN Mesh	WireGuard	No	No	No
GPU Passthrough	Yes	Yes	Yes	Yes
Live Migration	Yes	Yes	Yes	Yes
RBAC	3-tier	3-tier	Keystone	No
Audit Logging	Yes	Yes	Yes	No
Written in	Rust	Perl/C	Python	C
Memory Safety	Guaranteed	No	N/A	No
Setup Time	5 min	2 hours	2 days	Manual
License	MIT	AGPL	Apache 2.0	LGPL

Technology Stack

Layer	Technology
Language	Rust (2021 edition)
Async Runtime	Tokio 1.44
Web Framework	Axum 0.8
Terminal UI	ratatui + crossterm
Web UI	React 18 + TypeScript + Vite + TailwindCSS
VM Backend	systemd-vmspawn + systemd-machined
D-Bus	zbus 4
Monitoring	Prometheus

Project Statistics

Metric	Value
Backend crates	40
Rust source files	165
TypeScript source files	130
Total lines of code	~87,000
REST API endpoints	480+
WebSocket endpoints	3
Web pages	37+
Security audit rounds	9
Test suite	Passing

License

MIT — free for commercial use, modification, and distribution.

Getting Started

```bash
# Build
cd backend && cargo build --release

# Install
sudo make install

# Start
sudo systemctl enable --now vmspawnd

# Read admin password
sudo cat /var/lib/vmspawnd/.admin_password

# Login
curl -X POST http://localhost:8080/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username": "admin", "password": "<password>"}'

# Create your first VM
vmctl create myvm --image=/path/to/image.qcow2 --cpus=4 --memory=4096
vmctl start myvm

# Open web dashboard
open http://localhost:8080
```

For technical details, see the Architecture Guide, API Reference, and Security Documentation.